Lab-05: JWT Authentication and Authorization in ASP.NET Core Web API

Scenario

The objective of this exercise is to secure the ASP.NET Core Web API using JSON Web Token (JWT) authentication. The application generates JWT tokens, validates incoming requests using the Authorization header, verifies user roles, and checks token expiration.

Objective

- Configure JWT Authentication in ASP.NET Core Web API.
- Generate JWT tokens using an AuthController.
- Secure Web API endpoints using the **Authorize** attribute.
- Test authenticated requests using Postman.
- Validate JWT expiration.
- Implement role-based authorization.

Project Name

FirstWebAPI

Implementation

Step 1: Open Existing Project

Opened the existing **FirstWebAPI** project created in the previous labs.

Step 2: Install JWT Package

Installed the JWT authentication package using NuGet Package Manager.

Install-Package Microsoft.AspNetCore.Authentication.JwtBearer

Step 3: Configure JWT Authentication

Configured JWT Authentication in **Program.cs**.

The following settings were configured:

- Security Key
- Issuer
- Audience
- Token Validation Parameters
- Authentication Middleware

Authentication middleware was enabled using:

```
app.UseAuthentication();
app.UseAuthorization();
```

Step 4: Create AuthController

Created a new controller named **AuthController** inside the **Controllers** folder.

Added the **AllowAnonymous** attribute to allow users to generate JWT tokens without authentication.

Implemented the **GenerateJSONWebToken()** method.

The JWT token includes:

- UserId
- User Role
- Issuer
- Audience
- Expiration Time
- Signing Credentials

The GET action method invokes **GenerateJSONWebToken()** and returns the generated JWT token.

Step 5: Secure EmployeeController

Removed the **CustomAuthFilter** from **EmployeeController**.

Added the **Authorize** attribute.

[Authorize(Roles = "Admin")]

This ensures that only authenticated users with the **Admin** role can access the Employee API.

Step 6: Build the Project

Built the application successfully using:

Build → Rebuild Solution

Step 7: Execute the Application

Executed the application using:

Ctrl + F5

Swagger UI opened successfully.

Step 8: Generate JWT Token

Executed the following endpoint:

GET /api/Auth

The API generated a valid JWT token.

Step 9: Test Authentication Using Postman

Opened **Postman**.

Created a GET request.

Request URL:

https://localhost:xxxx/api/Employee

Added the Authorization header.

Key : Authorization

Value : Bearer <Generated JWT Token>

Executed the request.

The Employee API returned the employee list successfully.

Response Status:

200 OK

Step 10: Test Unauthorized Request

Executed the Employee API without the Authorization header.

Response:

401 Unauthorized

Step 11: Test Invalid JWT Token

Modified the generated JWT token.

Executed the Employee API again.

Response:

401 Unauthorized

Step 12: Test JWT Expiration

Modified the token expiration time to **2 minutes**.

Generated a new JWT token.

Waited for more than **2 minutes**.

Executed the Employee API using the expired token.

Response:

401 Unauthorized

Step 13: Test Role-Based Authorization

Modified the Authorize attribute.

[Authorize(Roles = "POC")]

Executed the Employee API using a token generated with the **Admin** role.

Response:

401 Unauthorized

Modified the Authorize attribute.

[Authorize(Roles = "Admin,POC")]

Executed the Employee API again.

The API returned the employee list successfully.

Response:

200 OK

Result

JWT Authentication was successfully implemented in the ASP.NET Core Web API. The application generated secure JWT tokens using the **AuthController**, authenticated requests using the **Authorize** attribute, validated user roles, and enforced token expiration. The API was successfully tested using **Swagger** and **Postman**, confirming proper authentication and authorization behavior.